

DigiBook AI: Project Documentation

Adaptive AI Textbook & Learning Platform Built by Ansh Saini, South Brunswick High School, New Jersey.
Submitted to ReverieHacks 2026, Software Development track.

Table of contents

1. Purpose
 2. Target audience
 3. Main features
 4. How it works: technical overview
 5. Installation and deployment
 6. Configuration
 7. User manual
 8. Accessibility guide
 9. Keyboard shortcuts and voice commands
 10. Data, storage, and privacy
 11. Troubleshooting
 12. Known limitations
 13. References and credits
-

1. Purpose

A textbook is a dead end. When a student hits a sentence they don't understand, the book has nothing more to offer, so they open a new tab, type a question that strips away all the surrounding context, and lose their place. The reading, the explanation, the practice, and the record of what they actually learned all live in different places, and none of them talk to each other.

DigiBook closes that loop. It takes an ordinary document (a PDF, a Word file, a plain text file) and turns it into an active lesson. The student can ask about any word or passage in place, get tutored rather than told, generate flashcards from what they just read, and watch a visible, honest model of which concepts they have actually mastered.

Two ideas from intelligent-tutoring-systems research carry the design:

- **Socratic tutoring:** the tutor asks guiding questions before it gives answers, because being handed an answer produces the weakest kind of learning.
- **Open student modeling:** the system's estimate of what you know is shown to you, not hidden. Every score is earned through your own flashcard reviews; nothing is ever pre-filled.

DigiBook extends earlier ARGOS research on Socratic tutoring and open student modeling into a practical, everyday reading tool.

DigiBook is an educational proof of concept. AI-generated explanations, translations, flashcards, and feedback are meant to support learning and should not replace textbooks, teachers, or trusted academic resources.

2. Target audience

Primary: students aged roughly 13-22 reading assigned material independently: high school and early undergraduate. Anyone with a chapter to get through and no one nearby to ask.

Specific groups the design serves deliberately:

Audience	What DigiBook gives them
Students with dyslexia	Easy-reading font, reading ruler, read-aloud
Students with low vision	Large / extra-large text, high contrast, zoom-friendly layout
Students with motor differences	Large touch targets, full keyboard control, optional drawing canvas
Students with attention differences	Reading ruler, reduced motion, Focus mode
English learners	Read-aloud paired with nine-language translation and explanation
Screen-reader users	Announced page turns, flashcard feedback, and study-room activity
Study pairs and small groups	Study Rooms with shared page, notes, sketch, chat, and voice
Teachers and tutors	Exportable study report showing concept mastery

Secondary audience: anyone reading technical material outside a classroom; the tool has no concept of a course, an enrollment, or a grade.

3. Main features

Reading

- Open **PDF, DOCX, or TXT** by drag-and-drop or file picker.
- **Demo Library**: 17 built-in lessons across Biology, Physics, Math, Computer Science, Space Exploration, Scientific Research, and Practical Skills. No upload or account needed. Demo lessons start with **zero mastery**; progress is earned, never pre-populated.
- Bookmarks, paragraph highlighting, three reading themes (light, sepia, dark), adjustable document text size, and a recently-opened shelf.
- **Read aloud** in the language the text is actually written in (a Hindi passage is spoken in Hindi, not in mispronounced English) with male/female voice and 0.75x–1.5x speed. The read-aloud bar stays hidden until you toggle it with the speaker button in the reader toolbar.

Learning

- **Select to learn**. Highlight any word or passage and choose **Define**, **Explain simply**, **Translate**, **Summarize**, **Ask tutor**, or **Add to Notes**, in any of nine languages.
- **Explanation level**: Simple, Standard, or Advanced.
- **Socratic tutor** with two modes: **Guide me** (asks guiding questions so the student reasons to the answer) and **Just answer** (direct response).

Flashcards: the practice engine

- Cards are generated by AI from the current page or from the student's own notes; each card is tagged with the **concept it tests**.
- **Ratings move mastery**. "Got it" raises that concept's score by 12; "Again" lowers it by 10 (clamped 0–100). Practice and progress are one system.
- **Leitner spaced repetition** on top: "Got it" advances a card a box (10 min → 1 day → 3 days → 7 days → 14 days); "Again" resets it to now. A due counter and "review due first" keep the deck honest.
- If the AI is unreachable, cards fall back to a **built-in extractive analyzer** (key terms + defining sentences from the actual page), always labeled as offline content.

The open student model

- **Mastery**: a visible bar for every assessed concept, banded as Strong (≥ 75), Developing (50–74), or Needs review (< 50), weakest first. Concepts you've merely encountered (looked up, asked about) appear unscored until a flashcard review assesses them.
- **Stats**: page views, time reading, words looked up, passages explained, cards reviewed, recall accuracy, and strong vs. needs-review counts.
- **Study report**: a printable summary of mastery, plus JSON export/import to move progress between devices.

Notebook: its own panel

- Opens on the **left side** and can stay open **alongside the tutor sidebar**: read in the middle, notes on the left, tutor on the right.
- **Notes mode**: type or dictate; passages added from the reader carry a *Go to Source* link back to the exact page; ask DigiBook to explain or check your notes; turn notes into flashcards with one button.
- **Sketch mode**: pen, highlighter, and eraser for equations and diagrams.
- Autosaves per book; export notes or sketches from the $\cdots$ menu.

Study Rooms

- **Create a room** from the top bar: room details appear in a centered window with a share code and link.
- Follow the presenter's page, take **collaborative notes and sketches** attributed to whoever wrote them, chat, and talk over **live voice**.
- Per-participant **edit / view-only** permissions.
- Private notebooks and progress are never shared; only the room's content.

Accessibility

A dedicated panel (the accessibility figure in the top bar) covering read-aloud, voice commands, dictation, high contrast, text size, spacing, dyslexia-friendly font, a reading ruler that bolds the line you point at, reading themes, reduced motion, explanation language and level, document text size, and a keyboard reference. See [section 8](#).

4. How it works: technical overview

Stack

Layer	Technology
Frontend	A single <code>index.html</code> : vanilla JavaScript, no framework, no build step
AI	Google Gemini, reached through a Netlify serverless function
Document parsing	pdf.js 3.11.174 and mammoth 1.6.0, loaded from CDN, run in-browser
Collaboration	Firebase Realtime Database (state sync) + WebRTC (voice)
Speech	Web Speech API: <code>speechSynthesis</code> for read-aloud, <code>SpeechRecognition</code> for dictation and commands
Persistence	Browser <code>localStorage</code>
Hosting	Netlify static hosting + Netlify Functions

There is no bundler, no `package.json`, and no install step. The deployable artifact is the folder itself.

Repository layout

```
.
├─ index.html           # the entire application
├─ netlify.toml         # publish dir + functions dir
├─ netlify/
│   └─ functions/
│       └─ ai-proxy.js  # Gemini proxy: hides the API key
├─ README.md
└─ DOCUMENTATION.md    # this file
```

Layout model

Three independent regions: the **Notebook panel** (left, toggled from the reader toolbar), the **reader** (center), and the **learning sidebar** (right, toggled from the top bar) holding Learn / Flashcards / Progress. Any combination can be open; the Study Room appears as a centered overlay above them.

Request flow

```
Browser (index.html)
  | POST { system, user, json }
  ▼
/.netlify/functions/ai-proxy  ← GEMINI_API_KEY lives here, never in the browser
  | tries models in order
  ▼
Google Generative Language API
```

The proxy tries a chain of models until one answers, and logs which model served each request (`[ai-proxy]` served by ...). The default chain is `gemini-2.5-flash` → `gemini-2.0-flash` → `gemini-flash-latest` → `gemini-2.5-flash-lite`, overridable without code changes via the `GEMINI_MODELS` environment variable (see [Configuration](#)).

Reliability details that matter:

- **Thinking is disabled** for models matching `2.5` or `-latest`, because "thinking" models can burn the whole output budget and return empty text.
- **Structured responses are validated server-side.** When the browser asks for JSON (flashcard generation), the proxy parses the model's output before accepting it; prose or truncated JSON causes a fall-through to the next model instead of a silent client-side failure. JSON calls get a 4,096-token budget.
- The browser retries up to 4 times with backoff (1.2s → 3s → 6s) tuned to free-tier per-minute limits, and logs the real upstream reason on failure.

Mastery model

Mastery is earned exclusively through flashcard self-ratings:

- A concept enters the model at **50** the first time a card assessing it is rated (demo lessons no longer pre-seed any scores).
- **"Got it" = +12, "Again" = -10**, clamped to 0-100.
- Bands: Strong ≥ 75 · Developing 50-74 · Needs review < 50 .

Each card knows its concept three ways, in priority order: an explicit `concept` tag from AI generation, a short term-style front used verbatim, or a match against the lesson's known concept pool. It is a deliberately simple additive model: legible to students, not statistically calibrated.

Spaced repetition

Six-box Leitner schedule per card:

Box	0	1	2	3	4	5
Next review	immediately	10 min	1 day	3 days	7 days	14 days

Honest-fallback design

Every external dependency can fail without lying about it. If the AI is unreachable, flashcards fall back to a built-in extractive analyzer over the actual page text, always labeled offline; the tutor posts a clearly marked "(Tutor offline)" message and preserves your question rather than inventing an answer. If pdf.js or mammoth fails to load from CDN, only that file format is disabled, with an explicit message.

5. Installation and deployment

Run locally

There is nothing to install and nothing to build. Any static file server works:

```
python3 -m http.server 8080
```

Then open `http://localhost:8080`.

Important: the AI tutor cannot work on a plain static server, because `/.netlify/functions/ai-proxy` only exists on Netlify. Reading, themes, accessibility, notes, sketching, offline flashcards, and all 17 demo lessons still work. To run the AI locally, use the Netlify dev server, which serves the function too:

```
npm install -g netlify-cli
```

```
GEMINI_API_KEY=your_key_here netlify dev
```

Deploy to Netlify

Option A: drag and drop

1. Zip this folder.
2. Drop the zip on the Netlify dashboard.
3. Go to **Site configuration** → **Environment variables** and add `GEMINI_API_KEY`.
4. Redeploy (**Deploys** → **Trigger deploy**). Env vars are read at deploy time.

Option B: CLI

```
netlify deploy --prod
```

Verify the deployment

Open `https://<your-site>/.netlify/functions/ai-proxy` in a browser. A `GET` returns a self-check:

```
{
  "ok": true,
  "keyConfigured": true,
  "models": ["gemini-2.5-flash", "gemini-2.0-flash", "gemini-flash-latest", "gemini-2.5-flash-lite"],
  "note": "Function is deployed. POST here to generate."
}
```

If `keyConfigured` is `false`, the environment variable is missing or the site has not been redeployed since it was set.

6. Configuration

Required: Gemini API key

Variable	Where	Value
GEMINI_API_KEY	Netlify environment variables	A Google AI Studio key; the free tier is sufficient

Get one at aistudio.google.com. The key is read only inside `netlify/functions/ai-proxy.js` and never reaches the browser.

Optional: model chain override

Variable	Example
GEMINI_MODELS	gemini-3-flash,gemini-flash-latest,gemini-2.5-flash,gemini-2.5-flash-lite

Comma-separated, highest priority first. Lets you adopt newer models from the Netlify dashboard without touching code. Requires a redeploy to take effect. The proxy's GET self-check shows the live chain.

Optional: Study Rooms (Firebase)

Study Rooms use a Firebase Realtime Database with anonymous auth. A working configuration ships in `index.html` (search for `FIREBASE_CONFIG`). To use your own project: create one at console.firebase.google.com, enable Realtime Database and Anonymous authentication, and replace the config object.

A Firebase web config, including `apiKey`, is a public client identifier by design, not a secret. What protects the data is your **database security rules**; if you deploy your own instance, restrict reads and writes to authenticated users and their room paths rather than leaving test mode on.

If Firebase is unreachable, Study Rooms are disabled and nothing else is affected.

Optional: featured demo

`index.html` defines `const DEFAULT_DEMO = 'genetics'`. Set it to any topic id: `photosynthesis`, `genetics`, `cell-structure`, `motion-forces`, `energy`, `electricity`, `algebra`, `functions`, `probability`, `ml-basics`, `algorithms`, `data-structures`, `mars-rover`, `black-holes`, `scientific-method`, `vaccines`, `carpentry-safety`.

Browser requirements

Feature	Requirement
Core reading, flashcards, notes	Any current browser
Read aloud	<code>speechSynthesis</code> ; all current browsers, voices vary by OS
Voice commands / dictation	Chrome and Edge; not available in Firefox
Study Room voice	WebRTC + microphone permission over HTTPS

Chrome or Edge is recommended for the full feature set.

7. User manual

The interface at a glance

Top bar, left to right: the DigiBook logo (click it to return home), the current document title, then three buttons: **Accessibility** (the figure icon), **Tutor & learning tools** (the chat icon, which opens the right sidebar), and **Create a room** (the people icon).

Reader toolbar: page arrows, bookmark, highlighter, the **speaker button** (shows/hides the read-aloud bar), and the **Notebook button** (opens the left panel). Exit returns to the library.

Getting started

1. **Open something to read.** Drag in a PDF, DOCX, or TXT, use the picker, or choose a lesson from **Explore Demo Library**.
2. **Open the learning tools** with the chat icon in the top bar. The sidebar has three tabs: **Learn**, **Flashcards**, and **Progress**, plus an **Expand** button that gives the sidebar the whole window.
3. **Set your language and level** in the Accessibility panel: explanation language (nine options) and explanation level (Simple / Standard / Advanced).

Asking about the text

Select a word or passage. A popup offers six actions:

Action	What you get
Define	A short definition of the selected term
Explain simply	The passage restated at your chosen level
Translate	The passage in your chosen language
Summarize	The key points of the passage
Ask tutor	Opens the tutor chat with the passage as context
Add to Notes	Sends the passage to the Notebook with a link back to this page

Using the tutor

The tutor lives in the **Learn** tab. Two modes:

- **Guide me** (default): the tutor asks you questions until you reach the answer yourself. Use this when you're studying.
- **Just answer:** a direct response, for when you only need a fact.

Type or use the microphone to dictate. If the AI is unreachable, the tutor says so plainly and keeps your question in the chat for when it's back.

Studying with flashcards

Open the **Flashcards** tab and generate cards from the current page, or from your notes via the Notebook's **Make Flashcards** button. For each card, flip it, then rate yourself:

- **Got it:** the card's concept gains mastery (+12) and the card returns later (10 min → 1 day → 3 days → 7 days → 14 days).
- **Again:** the concept loses mastery (−10) and the card comes back soon.

Review N due cards first sorts anything overdue to the front. This is the only way scores change: practice and progress are the same act.

Reading your progress

Progress → Mastery lists every assessed concept with a percentage and band, weakest first, plus unscored concepts you've encountered. **Progress → Stats** shows reading analytics (page views, time, lookups, passages explained) and learning analytics (cards reviewed, recall accuracy, strong vs. needs-review), plus:

Button	What it does
Export progress	Downloads a JSON file of concepts, scores, cards, and analytics
Import progress	Loads that file on another device or browser
Study report	Generates a printable mastery report
Reset all progress	Clears everything on this device (asks first)

Because there are no accounts, **export is the only way to move between devices.**

The Notebook

The Notebook button opens a **left-side panel that can stay open alongside the tutor**: notes on the left, reading in the middle, tutor on the right.

- **Notes**: type or dictate. Passages added from the reader carry a *Go to Source* link. Ask DigiBook to explain or check a note, or press **Make Flashcards** to turn your notes into cards.
- **Sketch**: pen, highlighter, eraser, thickness and color options, undo/redo.
- Scope notes to **this page** or **the whole book**. Everything autosaves per book; export from the ... menu.

Study Rooms

Press **Create a room** in the top bar. Room details open in a centered window: enter your name and create, or join with a friend's six-character code (or their link). Close the window with the ✕; the room stays active in the background.

Inside a room: follow the presenter's page or read at your own pace, add shared notes and sketches (attributed to their author), chat, talk over live voice, and share the current page, a note, or a sketch into the room. The host can set each participant to **Can edit** or **View only**. Your private notebook, mastery, and analytics are never shared.

Read aloud

Press the **speaker button** in the reader toolbar to show the read-aloud bar: play/pause, previous/next paragraph, stop, speed (0.75x–1.5x), and voice choice. Reading follows the language of the text. Turning on **Read aloud** in the Accessibility panel also adds per-message speaker buttons to tutor answers.

8. Accessibility guide

Accessibility is built into the reading experience rather than bolted on as a separate simplified mode. There is one app, and it adapts.

Open the panel with the **accessibility figure** in the top bar.

Vision and reading

- **Read aloud:** male or female voice, 0.75x–1.5x speed, spoken in whichever of the nine supported languages the text is written in.
- **Auto-read tutor responses.**
- **High contrast** mode.
- **Text size** (interface): default, large, extra large, plus **document text size** (A– / A+) for the book text itself.
- **Reading spacing:** default, comfortable, spacious.
- **Easy-reading font:** a dyslexia-friendly typeface.
- **Reading ruler:** **bolds** the line you point at.
- **Reading themes:** light, sepia, dark.
- **Explanation level and language:** how simply, and in which of nine languages, the AI explains.

Motor and mobility

- **Easy interaction mode:** larger touch targets, visible button labels.
- **Voice commands:** drive the app by speaking (see below).
- Full keyboard operation with a visible focus outline; the sketch canvas is always optional.

Comfort

- **Reduce motion:** suppresses animation and transitions.

Screen reader support

- Live regions announce page turns, flashcard feedback, and Study Room activity.
- Skip links to the document and the tools appear on the first Tab press.
- All controls carry labels and pressed/expanded state.

Choices are saved per device. **Reset accessibility settings** restores defaults without touching learning progress.

9. Keyboard shortcuts and voice commands

Keyboard

Key	Action
Tab / Shift + Tab	Move between controls
Enter or Space	Activate the focused control
← / →	Previous / next document page
Esc	Close a menu or dialog
Tab at the top of the page	Reveal skip links

Voice commands

Turn on **Voice commands** in the Accessibility panel, then say:

Say	Does
"Open learn" / "Go to learn"	Learn tab
"Open flashcards" / "Flash cards" / "Open cards"	Flashcards tab
"Open progress"	Progress tab
"Open mastery" / "Show mastery"	Progress → Mastery
"Open stats" / "Statistics"	Progress → Stats
"Read page" / "Read aloud" / "Read this"	Read the current page
"Stop reading" / "Stop aloud"	Stop reading
"Next page" / "Previous page"	Turn pages
"Focus mode" / "Expand"	Expand the sidebar
"Back to reader" / "Split view"	Restore the split layout

Voice commands require Chrome or Edge.

10. Data, storage, and privacy

There are no accounts. Nothing to sign up for, nothing to sign in to.

What is stored, and where

Everything lives in this browser's `localStorage` under the `digibook_v1` prefix:

Key	Contents
<code>digibook_v1</code>	Concepts, mastery scores, flashcards, review schedule, analytics
<code>digibook_v1_acx</code>	Accessibility settings
<code>digibook_v1_nb</code>	Notebook notes and sketches
<code>digibook_v1_shelf</code>	Recently opened documents
<code>digibook_v1_theme</code> / <code>_level</code> / <code>_read</code>	Theme, explanation level, font choice

Clearing browser data clears all of it. Nothing syncs between devices; use **Export progress** to move it yourself.

What leaves the browser

- **Your document does not.** PDF, DOCX, and TXT parsing happens entirely in the browser. The file is never uploaded.
 - **Only the passage you ask about** is sent to the AI proxy, with your question, language, and level, not the document, notes, or scores.
 - **Study Room content** (shared page, notes, sketch, chat) syncs through Firebase while a room is active. Voice is peer-to-peer over WebRTC.
 - The Gemini API key and model choice stay server-side in the Netlify function.
-

11. Troubleshooting

The tutor says "(Tutor offline)". First: are you on a plain local server or `file:///` ? The AI **cannot** work there; the proxy function only exists on Netlify (or under `netlify dev`). If you're on the deployed site, open `https://<your-site>/.netlify/functions/ai-proxy` : if `keyConfigured` is `false` , set `GEMINI_API_KEY` and redeploy. Otherwise check **Logs → Functions → ai-proxy**: each failed model logs its reason (`HTTP 429` = quota, `HTTP 404` = model name not served to your key, `empty response` , `unparseable JSON`). The browser console also logs `[DigiBook AI]` lines.

Flashcards say "offline". Same cause: the cards were built by the local text analyzer instead of the AI. They're real content from the page, just simpler. Fix the AI connection above.

A PDF won't open. `pdf.js` loads from CDN. If it's blocked, that format is disabled with a message; DOCX, TXT, and demos still work. Check the console for a blocked `cdnjs.cloudflare.com` request.

Read aloud is silent, or uses the wrong voice. Voices come from the operating system. If no voice exists for the text's language, install that language pack at the OS level.

The microphone button does nothing. Voice input needs `SpeechRecognition` , which Firefox doesn't implement. Use Chrome or Edge and grant microphone permission.

Study Room voice doesn't connect. WebRTC needs HTTPS and microphone permission; some networks block peer connections.

Progress disappeared. `localStorage` is per-browser and per-device, and clearing site data erases it. There is no server copy, so export regularly if it matters.

12. Known limitations

- **No accounts and no sync.** Progress is device-local; export/import is manual.
 - **Voice input is Chromium-only.**
 - **AI output needs verification.** Explanations, translations, and generated cards can be wrong. The app says so in-product.
 - **Scanned PDFs without a text layer won't work:** there is no OCR.
 - **Mastery is a simple additive model** (+12 / −10), not calibrated knowledge tracing. It's legible and responsive, by design, but not statistically fitted.
 - **Concept tagging is heuristic** for cards that arrive without an explicit tag: good on the curated demos, rougher on arbitrary uploads.
 - **Study Rooms are small-group**, built for a study pair or a handful of students, not a lecture hall.
 - **Single-file architecture.** It deploys anywhere with zero tooling, which was the point, but it is not how you'd structure this for a team.
-

13. References and credits

Author. Ansh Saini, South Brunswick High School, New Jersey.

Research lineage. DigiBook extends earlier **ARGOS** work on Socratic tutoring and open student modeling. The ideas it builds on: guiding questions produce stronger learning than direct answers; showing learners the system's model of their knowledge supports self-regulated learning; and expanding review intervals (Leitner spaced repetition) produce durable retention.

Libraries and services.

Component	Used for	License / terms
pdf.js 3.11.174	In-browser PDF parsing	Apache-2.0
mammoth.js 1.6.0	In-browser DOCX parsing	BSD-2-Clause
Firebase 10.12.2	Study Room sync + anonymous auth	Google terms
Google Gemini API	Explanations, tutoring, flashcards	Google AI terms
Netlify	Static hosting + serverless functions	Netlify terms
Google Fonts	Typography	Open Font License
Web Speech API / WebRTC	Read-aloud, dictation, voice	Browser built-in

Demo lesson content was written for this project.

License. See `LICENSE` in the repository root.